RAPPORT D'ATELIER

Atelier 2

Procédures et Fonctions Stockées

Base de données MySQL • Aiven • Terminal (mysql.exe)

Réalisé par

Basma Elmaimouni

Module M203

11 novembre 2026

Sommaire

1	Connexion à MySQL sur Aiven
2	Gestion des stagiaires
3	Facturation de l'eau
4	Paramètres IN, OUT et INOUT
5	Suppression des procédures et fonctions
6	Conclusion

Environnement

Les commandes ont été exécutées dans le terminal Windows (cmd) avec le client **mysql.exe** de MySQL Workbench 8.0 CE, sur un service MySQL 8.4.11 hébergé sur **Aiven**, dans la base de données **defaultdb**.

1 Connexion à MySQL sur Aiven

Objectif Service MySQL créé sur Aiven, puis connexion depuis le terminal avec le client mysql.exe en SSL.

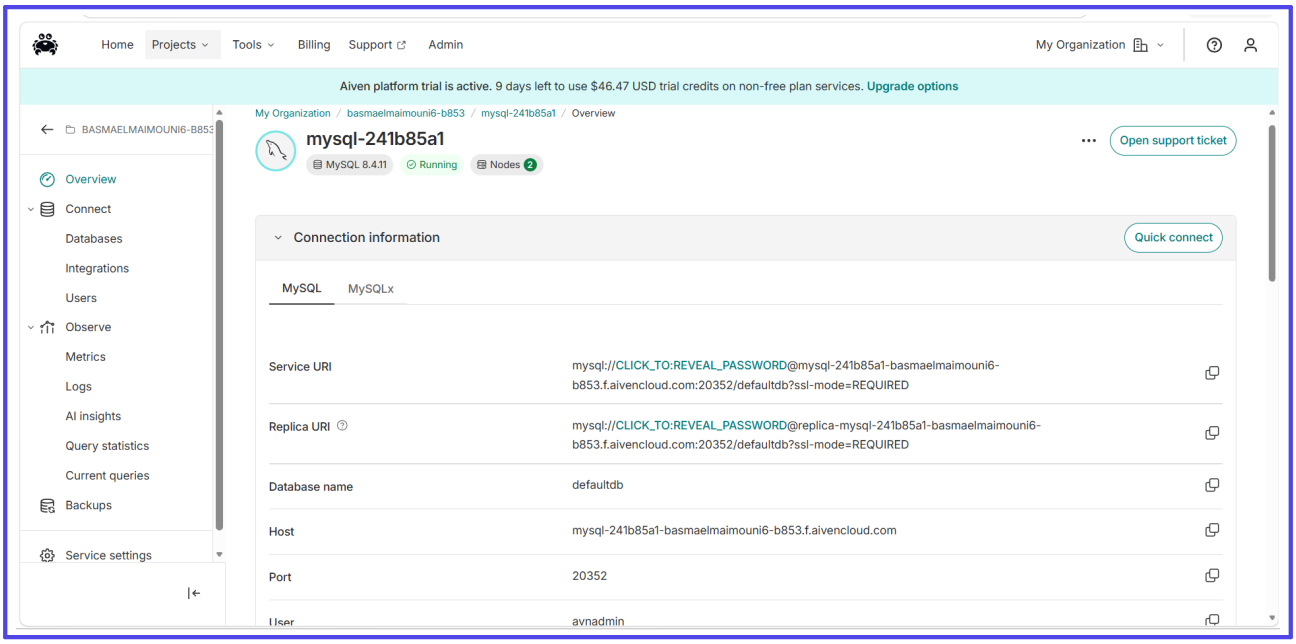


Figure 1.1 — Console Aiven : le service mysql-241b85a1 (MySQL 8.4.11) est en statut Running ; hôte, port 20352 et utilisateur avnadmin.

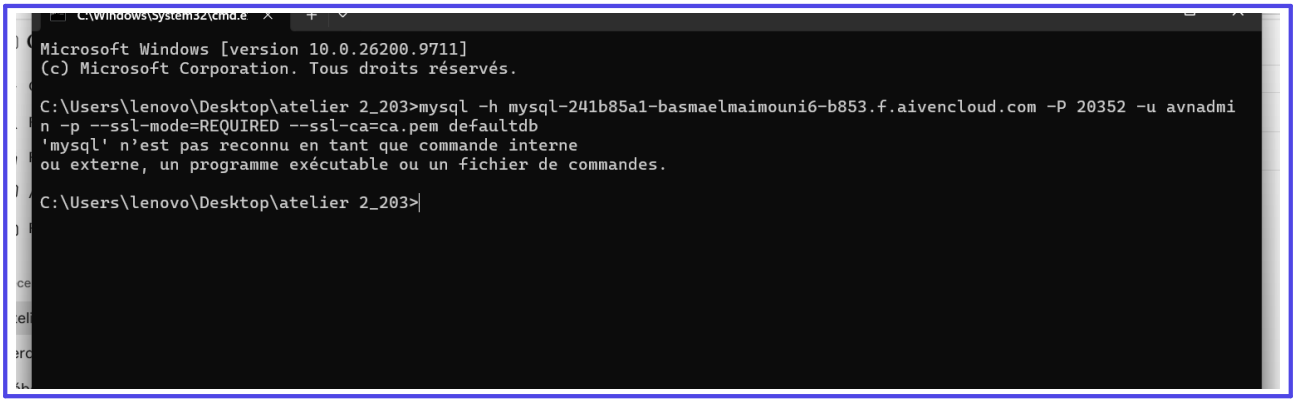


Figure 1.2 — La commande mysql n'est pas reconnue : le client n'est pas dans le PATH de Windows.

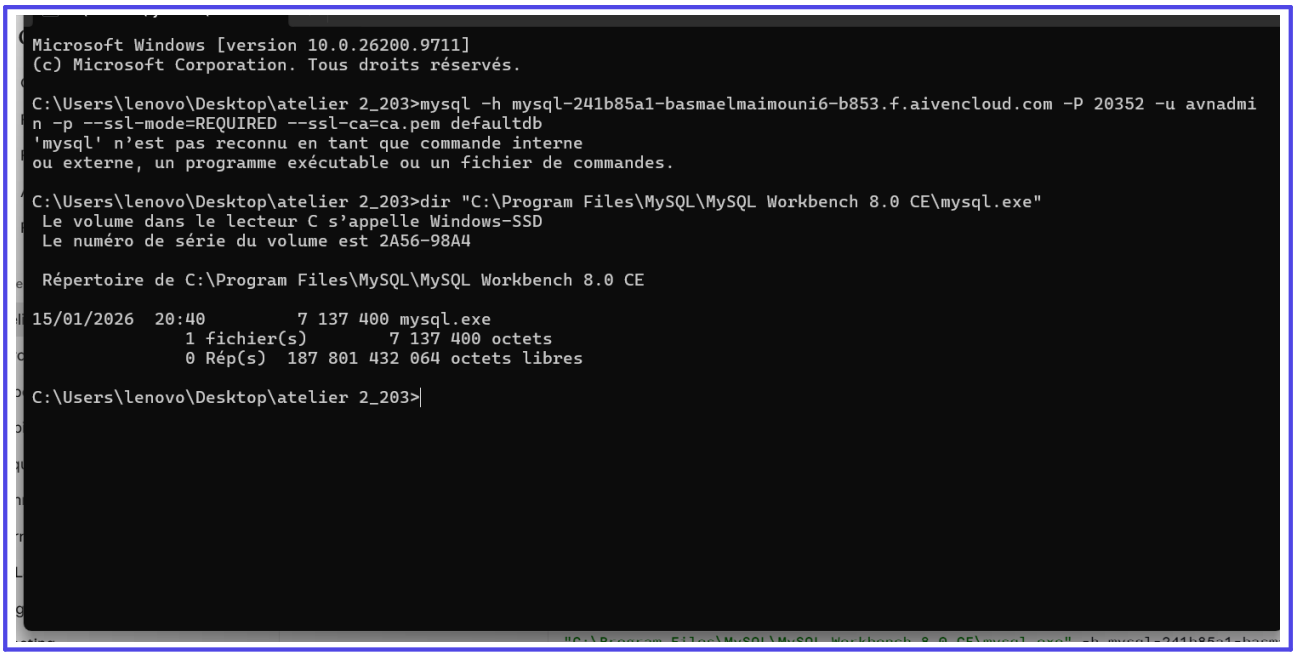


Figure 1.3 — Vérification avec dir : mysql.exe existe bien dans le dossier de MySQL Workbench 8.0 CE.

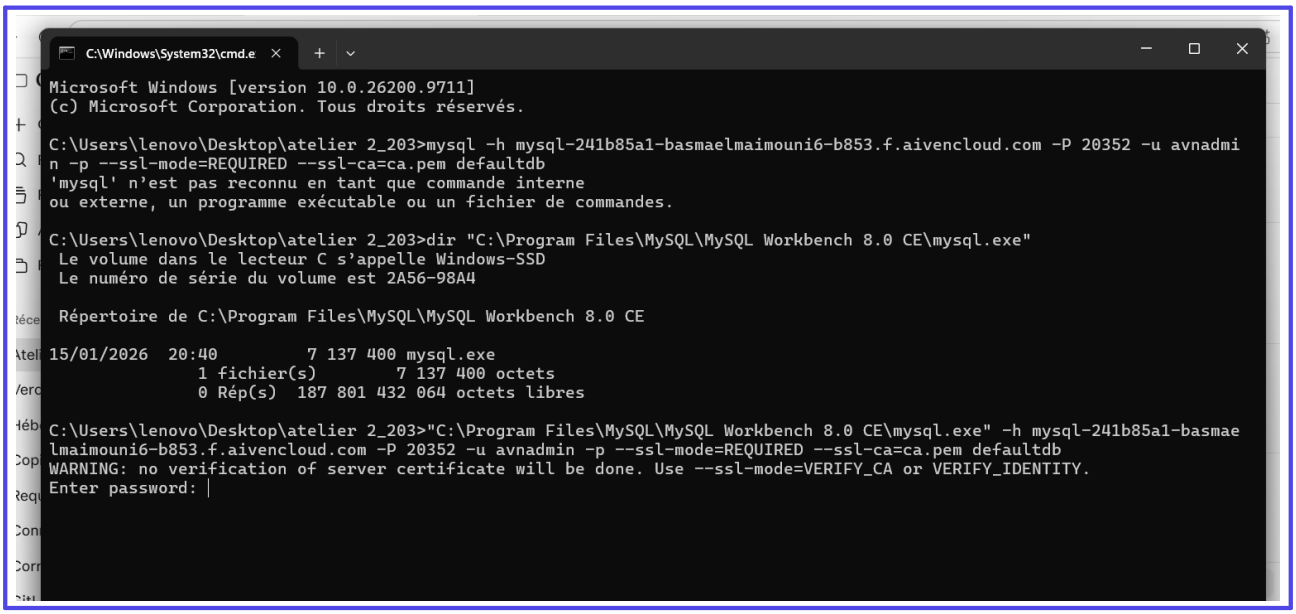


Figure 1.4 — Connexion avec le chemin complet de mysql.exe : l'avertissement SSL est sans gravité, le mot de passe est demandé.

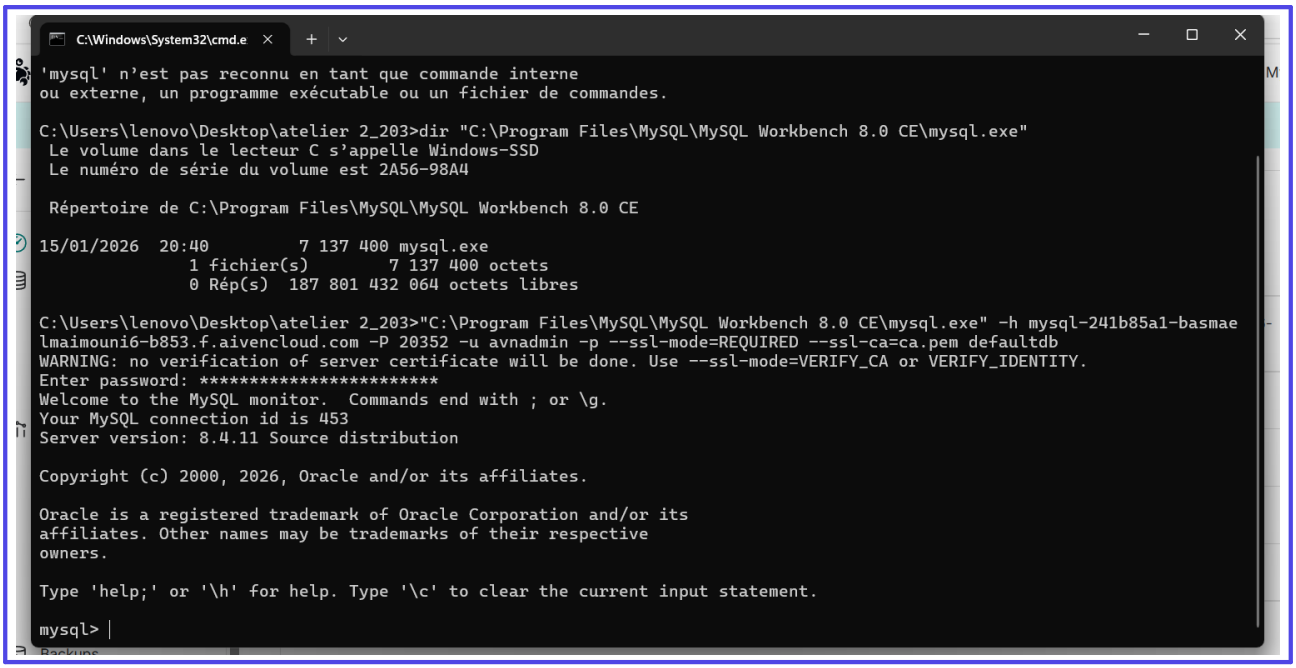


Figure 1.5 — *Connexion réussie : l'invite mysql> apparaît (connexion n° 453, serveur 8.4.11).*

2 Gestion des stagiaires

Objectif Création de la table stagiaires et de ses données, des procédures pc1, afficher_stagiaires_par_ville et compter_stagiaires, puis de la fonction nombre_stagiaires_par_ville.

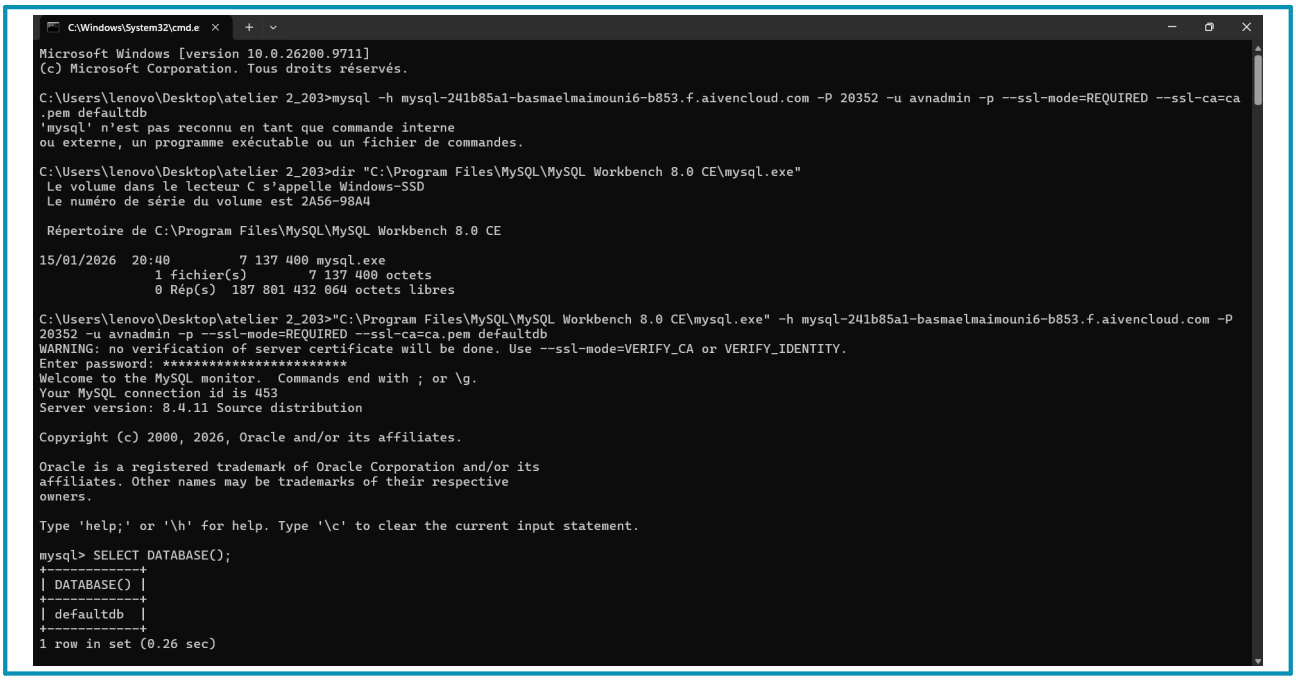


Figure 2.1 — Base courante vérifiée avec SELECT DATABASE() : defaultdb.

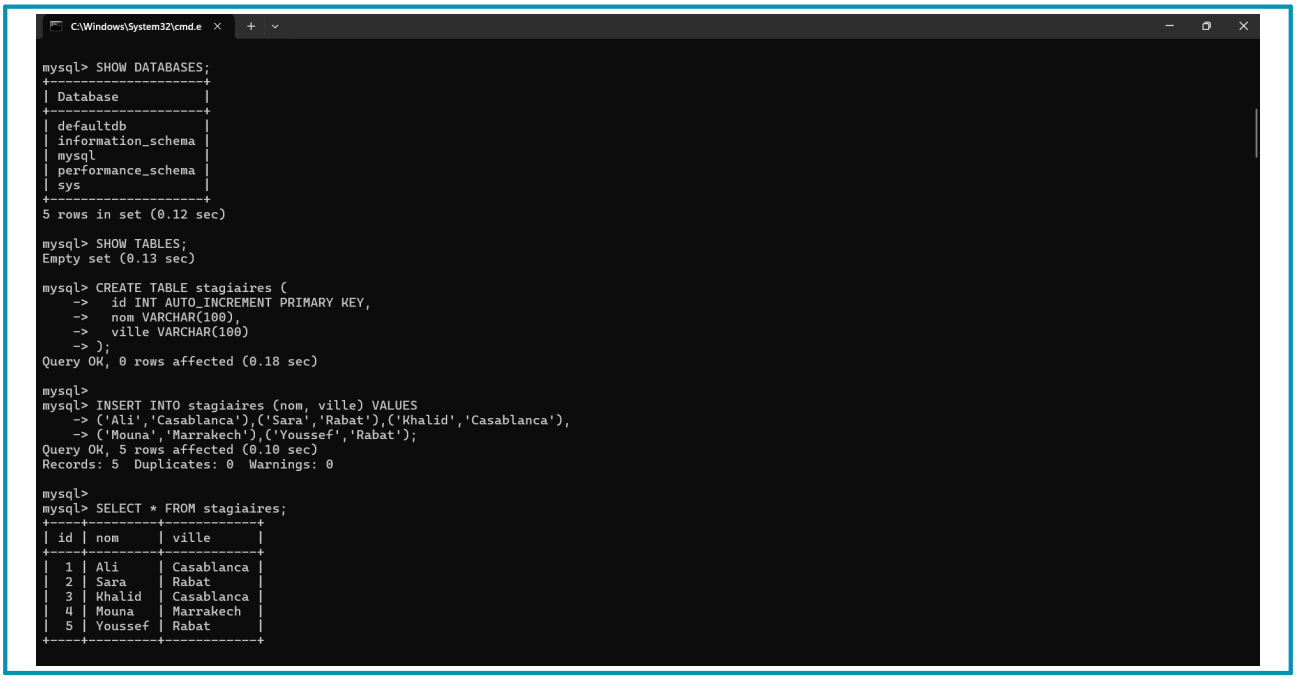
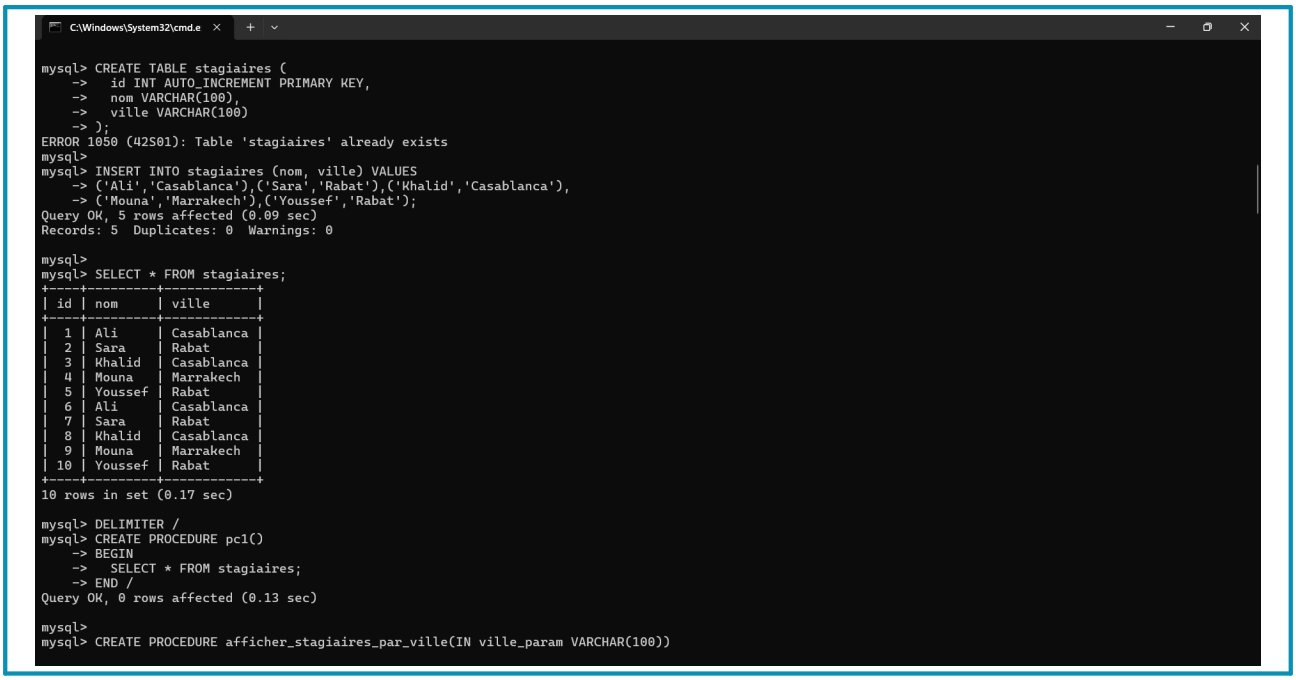


Figure 2.2 — SHOW DATABASES, SHOW TABLES (vide), création de la table stagiaires et insertion de 5 stagiaires.



```
C:\Windows\System32\cmd.e

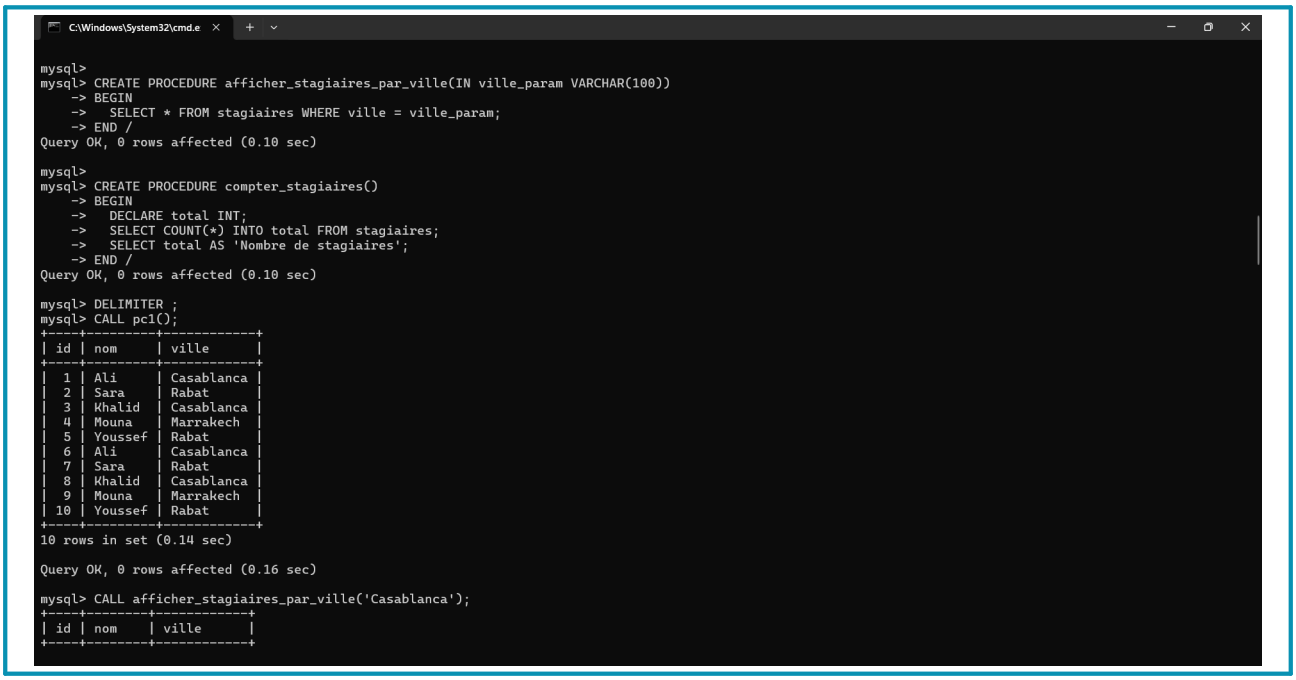
mysql> CREATE TABLE stagiaires (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   nom VARCHAR(100),
    ->   ville VARCHAR(100)
    -> );
ERROR 1050 (42S01): Table 'stagiaires' already exists
mysql>
mysql> INSERT INTO stagiaires (nom, ville) VALUES
    -> ('Ali','Casablanca'),('Sara','Rabat'),('Khalid','Casablanca'),
    -> ('Mouna','Marrakech'),('Youssef','Rabat');
Query OK, 5 rows affected (0.09 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM stagiaires;
+----+---------+------------+
| id | nom     | ville      |
+----+---------+------------+
|  1 | Ali     | Casablanca |
|  2 | Sara    | Rabat      |
|  3 | Khalid  | Casablanca |
|  4 | Mouna   | Marrakech  |
|  5 | Youssef | Rabat      |
|  6 | Ali     | Casablanca |
|  7 | Sara    | Rabat      |
|  8 | Khalid  | Casablanca |
|  9 | Mouna   | Marrakech  |
| 10 | Youssef | Rabat      |
+----+---------+------------+
10 rows in set (0.17 sec)

mysql> DELIMITER /
mysql> CREATE PROCEDURE pc1()
    -> BEGIN
    ->   SELECT * FROM stagiaires;
    -> END /
Query OK, 0 rows affected (0.13 sec)

mysql>
mysql> CREATE PROCEDURE afficher_stagiaires_par_ville(IN ville_param VARCHAR(100))
```

Figure 2.3 — *Le script a été collé une seconde fois : erreur 1050 (table déjà existante) et 10 lignes dans stagiaires ; création de pc1.*



```
C:\Windows\System32\cmd.e

mysql>
mysql> CREATE PROCEDURE afficher_stagiaires_par_ville(IN ville_param VARCHAR(100))
    -> BEGIN
    ->   SELECT * FROM stagiaires WHERE ville = ville_param;
    -> END /
Query OK, 0 rows affected (0.10 sec)

mysql>
mysql> CREATE PROCEDURE compter_stagiaires()
    -> BEGIN
    ->   DECLARE total INT;
    ->   SELECT COUNT(*) INTO total FROM stagiaires;
    ->   SELECT total AS 'Nombre de stagiaires';
    -> END /
Query OK, 0 rows affected (0.10 sec)

mysql> DELIMITER ;
mysql> CALL pc1();
+----+---------+------------+
| id | nom     | ville      |
+----+---------+------------+
|  1 | Ali     | Casablanca |
|  2 | Sara    | Rabat      |
|  3 | Khalid  | Casablanca |
|  4 | Mouna   | Marrakech  |
|  5 | Youssef | Rabat      |
|  6 | Ali     | Casablanca |
|  7 | Sara    | Rabat      |
|  8 | Khalid  | Casablanca |
|  9 | Mouna   | Marrakech  |
| 10 | Youssef | Rabat      |
+----+---------+------------+
10 rows in set (0.14 sec)

Query OK, 0 rows affected (0.16 sec)

mysql> CALL afficher_stagiaires_par_ville('Casablanca');
+----+--------+------------+
| id | nom    | ville      |
+----+--------+------------+
```

Figure 2.4 — *Création des procédures avec DELIMITER / puis CALL pc1() : 10 lignes affichées.*

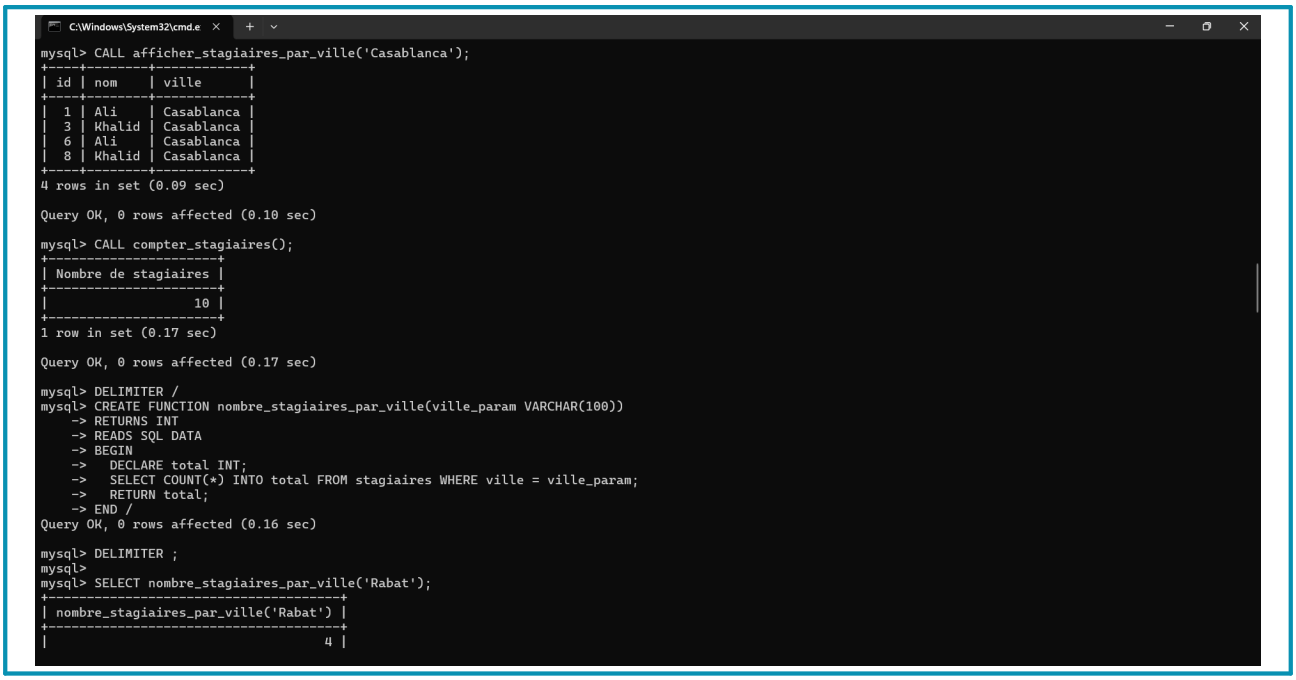


Figure 2.5 — CALL afficher_stagiaires_par_ville('Casablanca') et CALL compter_stagiaires() ; création de la fonction (READS SQL DATA) et SELECT nombre_stagiaires_par_ville('Rabat').

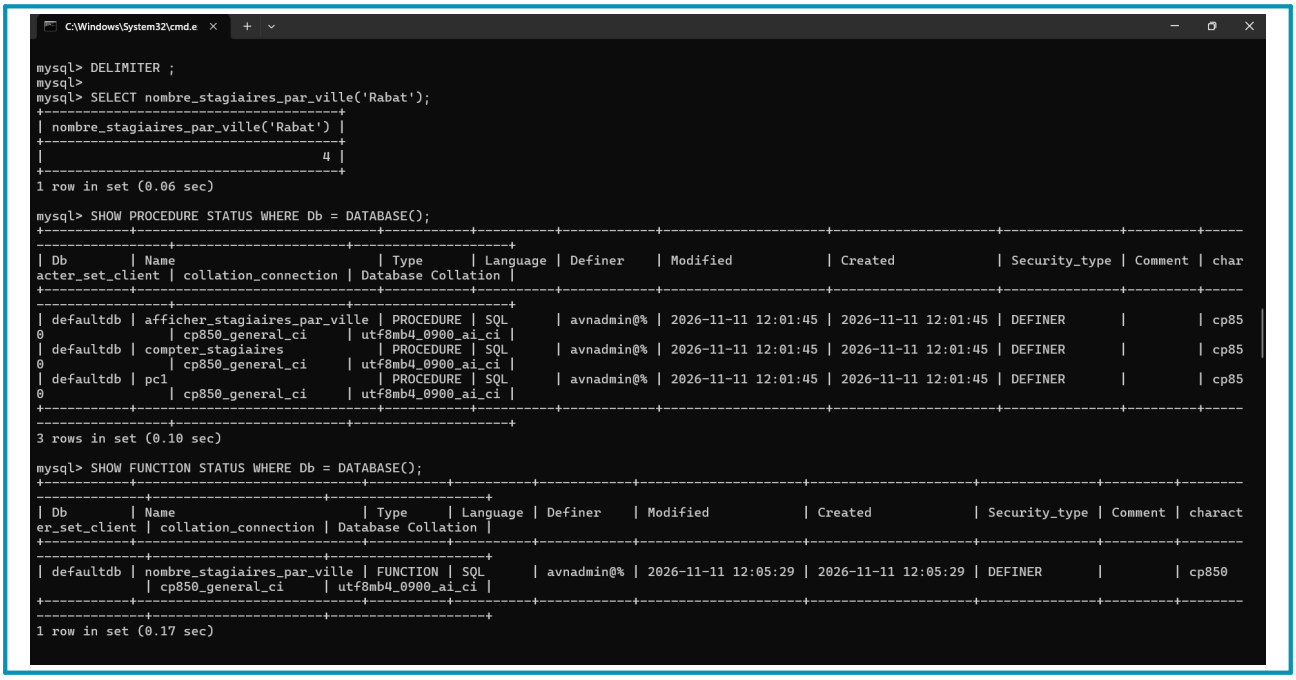


Figure 2.6 — SHOW PROCEDURE STATUS (3 procédures) et SHOW FUNCTION STATUS (1 fonction) dans defaultdb.

> **Remarque** Les résultats 10, 4 et 4 sont le double des valeurs attendues (5 stagiaires au total, 2 à Casablanca, 2 à Rabat) : l'INSERT a été exécuté deux fois (ids 6 à 10). Les procédures et la fonction fonctionnent correctement.

3 Facturation de l'eau

Objectif Table facture, procédure inserer_factures (boucle WHILE) et fonction calculer_montant (IF / ELSEIF) selon le barème : 1 € par m^3 jusqu'à 10 m^3, 1,5 € de 11 à 30 m^3, 2 € au-delà.



```
1 row in set (0.17 sec)

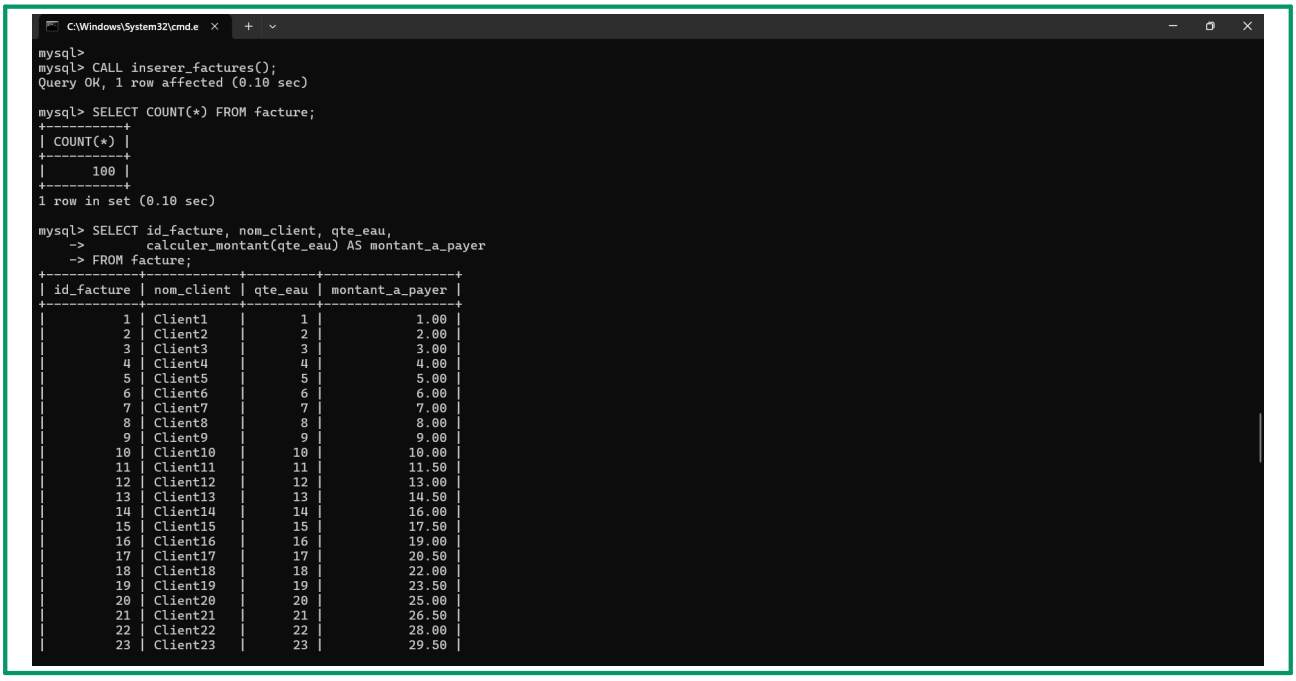
mysql> CREATE TABLE facture (
    ->   id_facture INT AUTO_INCREMENT PRIMARY KEY,
    ->   nom_client VARCHAR(255),
    ->   qte_eau INT
    -> );
Query OK, 0 rows affected (0.14 sec)

mysql>
mysql> DELIMITER /
mysql> CREATE PROCEDURE inserer_factures()
    -> BEGIN
    ->   DECLARE compteur INT DEFAULT 1;
    ->   WHILE compteur <= 100 DO
    ->     INSERT INTO facture (nom_client, qte_eau)
    ->     VALUES (CONCAT('Client', compteur), compteur);
    ->     SET compteur = compteur + 1;
    ->   END WHILE;
    -> END /
Query OK, 0 rows affected (0.10 sec)

mysql>
mysql> CREATE FUNCTION calculer_montant(qte INT)
    -> RETURNS DECIMAL(10,2)
    -> DETERMINISTIC
    -> BEGIN
    ->   DECLARE montant DECIMAL(10,2);
    ->   IF qte <= 10 THEN
    ->     SET montant = qte * 1;
    ->   ELSEIF qte <= 30 THEN
    ->     SET montant = (10 * 1) + (qte - 10) * 1.5;
    ->   ELSE
    ->     SET montant = (10 * 1) + (20 * 1.5) + (qte - 30) * 2;
    ->   END IF;
    ->   RETURN montant;
    -> END /
Query OK, 0 rows affected (0.09 sec)

mysql> DELIMITER ;
mysql>
```

Figure 3.1 — Création de la table facture, de la procédure inserer_factures et de la fonction calculer_montant.



```
mysql>
mysql> CALL inserer_factures();
Query OK, 1 row affected (0.10 sec)

mysql> SELECT COUNT(*) FROM facture;
+----------+
| COUNT(*) |
+----------+
|      100 |
+----------+
1 row in set (0.10 sec)

mysql> SELECT id_facture, nom_client, qte_eau,
    ->        calculer_montant(qte_eau) AS montant_a_payer
    -> FROM facture;
+------------+------------+---------+-----------------+
| id_facture | nom_client | qte_eau | montant_a_payer |
+------------+------------+---------+-----------------+
|          1 | Client1    |       1 |            1.00 |
|          2 | Client2    |       2 |            2.00 |
|          3 | Client3    |       3 |            3.00 |
|          4 | Client4    |       4 |            4.00 |
|          5 | Client5    |       5 |            5.00 |
|          6 | Client6    |       6 |            6.00 |
|          7 | Client7    |       7 |            7.00 |
|          8 | Client8    |       8 |            8.00 |
|          9 | Client9    |       9 |            9.00 |
|         10 | Client10   |      10 |           10.00 |
|         11 | Client11   |      11 |           11.50 |
|         12 | Client12   |      12 |           13.00 |
|         13 | Client13   |      13 |           14.50 |
|         14 | Client14   |      14 |           16.00 |
|         15 | Client15   |      15 |           17.50 |
|         16 | Client16   |      16 |           19.00 |
|         17 | Client17   |      17 |           20.50 |
|         18 | Client18   |      18 |           22.00 |
|         19 | Client19   |      19 |           23.50 |
|         20 | Client20   |      20 |           25.00 |
|         21 | Client21   |      21 |           26.50 |
|         22 | Client22   |      22 |           28.00 |
|         23 | Client23   |      23 |           29.50 |
```

Figure 3.2 — CALL inserer_factures() puis SELECT COUNT(*) : 100 factures ; début du résultat avec montant_a_payer.

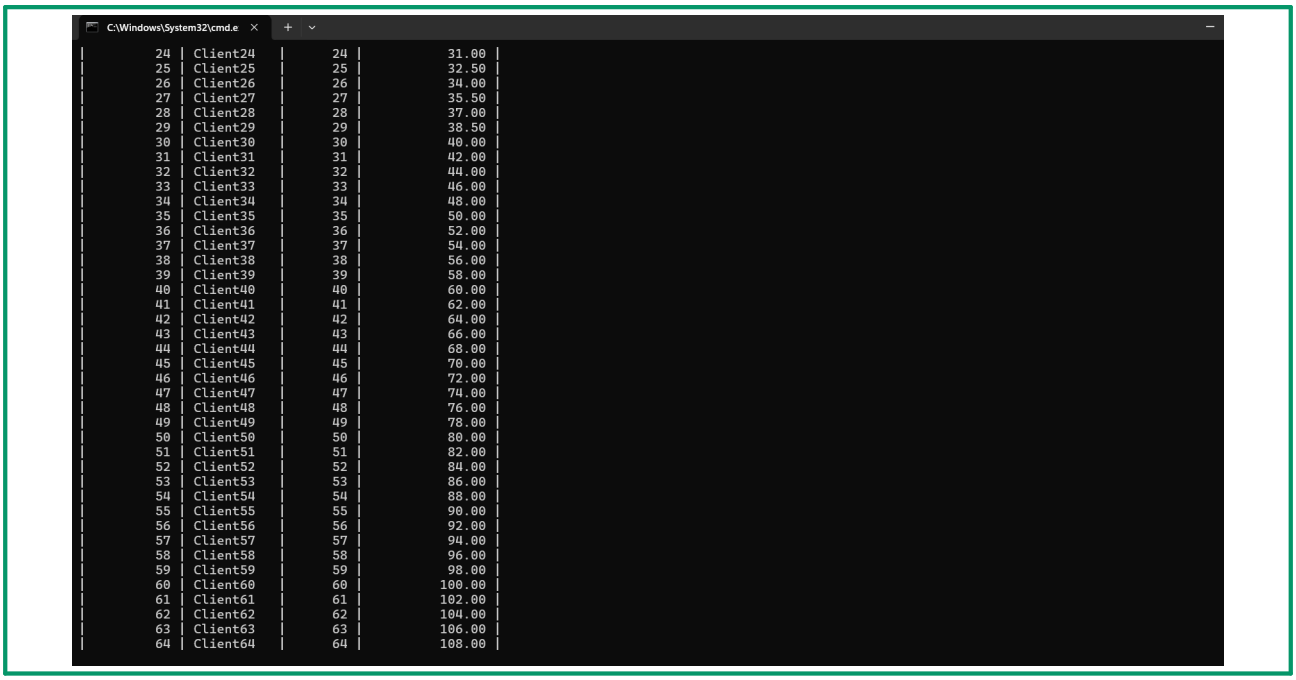


Figure 3.3 — *Suite du résultat : factures 24 à 64.*

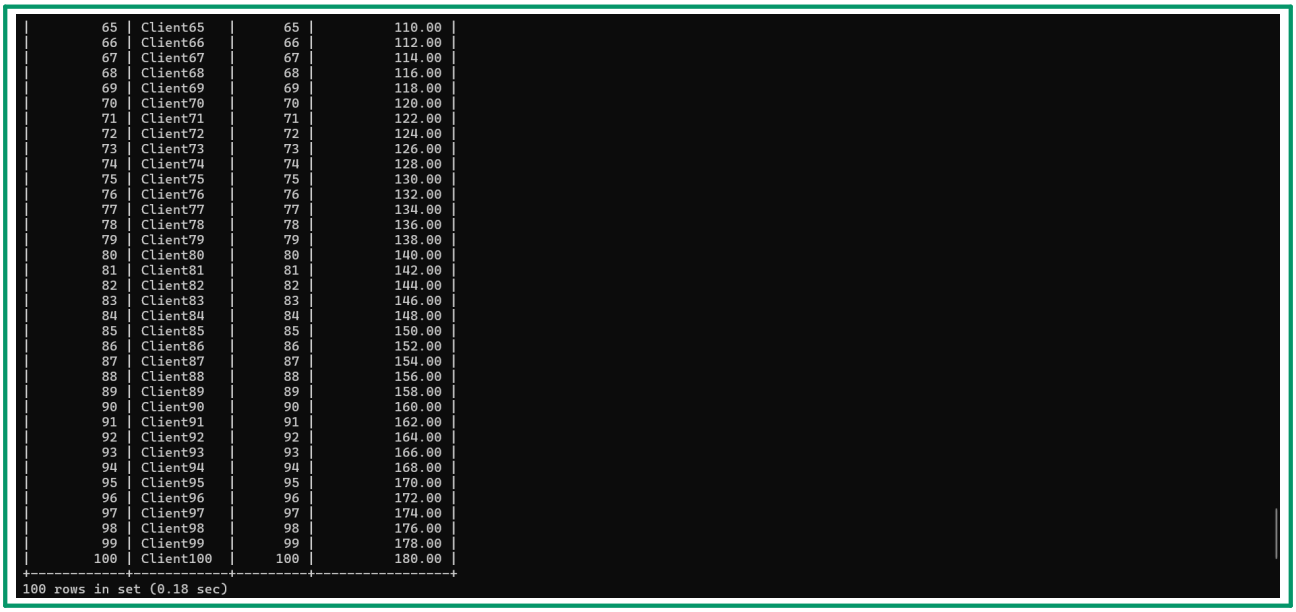


Figure 3.4 — *Fin du résultat : factures 65 à 100 (100 rows in set).*

Remarque Vérification avec le barème : 10 m^3 → 10.00 ; 30 m^3 → 40.00 ; 40 m^3 → 60.00 ; 100 m^3 → 180.00.

4 Paramètres IN, OUT et INOUT

Objectif Essai des trois types de paramètres : IN (supprimer_stagiaire), OUT (compter_stagiaires_out) et INOUT (augmenter_stock).

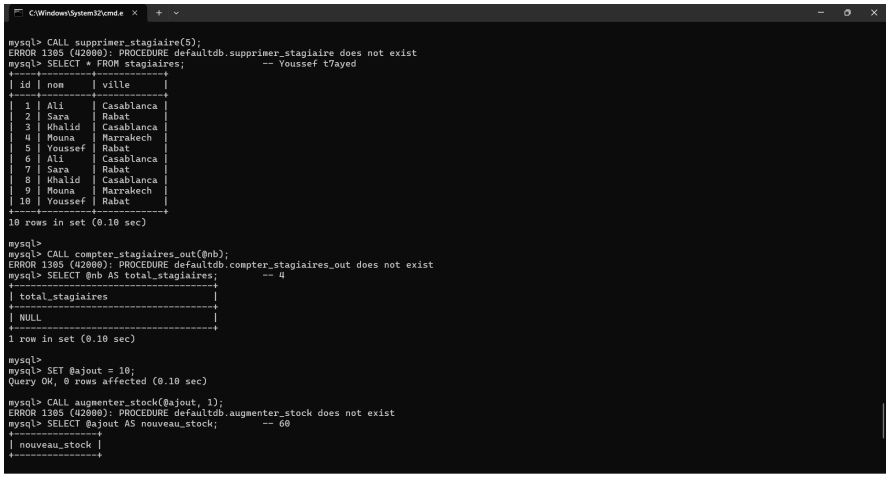


Figure 4.1 — Appels CALL supprimer_stagiaire(5), CALL compter_stagiaires_out(@nb) et CALL augmenter_stock(@ajout, 1) : erreur 1305, les procédures n'existent pas.

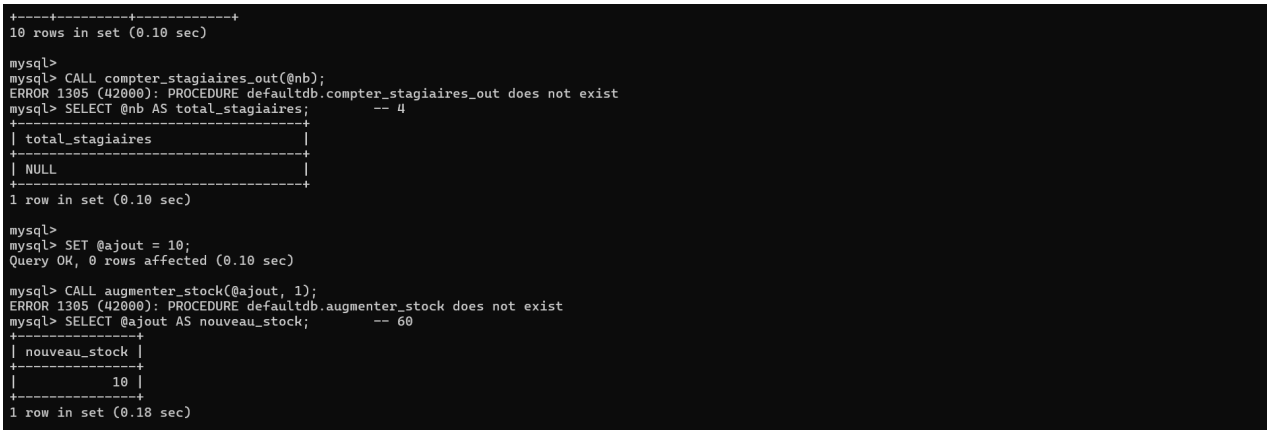


Figure 4.2 — Résultats observés : @nb vaut NULL et @ajout reste à 10 (valeur de départ).

Type	Direction	Rôle	Procédure
IN	Entrée	Envoyer une valeur	supprimer_stagiaire
OUT	Sortie	Récupérer une valeur calculée	compter_stagiaires_out
INOUT	Entrée / sortie	Envoyer puis modifier	augmenter_stock

Remarque Les trois procédures n'avaient pas été créées au moment des appels (seuls les CALL ont été exécutés, sans les CREATE PROCEDURE) : d'où l'erreur 1305. Le tableau ci-dessus rappelle le rôle de chaque type de paramètre.

5 Suppression des procédures et fonctions

Objectif Suppression des procédures et de la fonction de la partie 1 avec DROP … IF EXISTS.



Figure 5.1 — DROP PROCEDURE afficher_stagiaires_par_ville, DROP PROCEDURE compter_stagiaires et DROP FUNCTION nombre_stagiaires_par_ville : Query OK.

6 Conclusion

Cet atelier m'a permis de me connecter à une base MySQL hébergée sur Aiven depuis le terminal, puis de créer des procédures et des fonctions stockées. J'ai utilisé **DELIMITER** pour écrire des blocs complets, la boucle **WHILE** pour insérer 100 factures et **IF / ELSEIF** pour calculer un montant par tranches.

Une procédure s'appelle avec **CALL**, alors qu'une fonction retourne une valeur utilisable dans un **SELECT**. Sur Aiven, une fonction doit déclarer **READS SQL DATA** ou **DETERMINISTIC**. Enfin, une procédure doit être créée avant d'être appelée, sinon MySQL renvoie l'erreur 1305.